

Face Mask Detection System

Software Requirements Specification

Version 1.0



Group Id: F25PROJECTCC5EC

Supervisor Name: Madiha Faqir Hussain

Revision History

Date (dd/mm/yyyy)	Version	Description	Author
3-12-2025	1.0	This project aims to develop an automated face mask detection system using deep learning techniques. The system will identify whether a person in an image is wearing a mask or not wearing a mask. A Convolutional Neural Network (CNN) model will be trained on a dataset containing face images with and without masks. The model will be integrated with a real-time detection interface to monitor live camera feeds or uploaded images. This technology can be applied in public safety monitoring, hospital check-ins, and smart surveillance systems	BC220404931

Table of Contents

1. [Scope \(of the project\)](#)
2. [Functional Requirements Non Functional requirements](#)
3. [Use Case Diagram](#)
4. [Usage Scenarios](#)
5. [Adopted Methodology](#)
6. [Work Plan \(Use MS Project to create Schedule/Work Plan\)](#)

SRS Document

Scope of Project:

The scope of this project is to design and develop an automated **Face Mask Detection System** using deep learning and computer vision techniques. The system will detect whether a person in an image or video stream is wearing a face mask or not. It will use a trained Convolutional Neural Network (CNN) model to classify faces into two categories: **Mask** and **No Mask**. The solution will support both **image-based detection** and **real-time detection** through a webcam or live camera feed.

The project covers the complete workflow, including data collection, preprocessing, model training, user interface development, and database integration. A dataset of masked and unmasked face images will be gathered, preprocessed, and used to train an accurate and reliable CNN model. Face detection techniques such as OpenCV or MediaPipe will be used to locate faces, and the extracted regions will be normalized and fed into the model for prediction.

The system will provide a user-friendly interface where users can upload images for analysis or activate real-time mask detection. The interface will display results along with confidence scores and bounding boxes around detected faces. A backend database (SQLite or MySQL) will store detection results, including timestamps and prediction outcomes, allowing administrators to review and analyze system activity.

The project also includes performance evaluation under different lighting and camera conditions to ensure real-world usability. The model will be optimized for accuracy, speed, and stability, making the system suitable for applications in public safety monitoring, hospital entry points, campus surveillance, and other environments where mask compliance needs to be checked.

Functional and non Functional Requirements:

Functional Requirements

FR1 — Data Collection & Preprocessing

- 1.1 Collect or create a dataset containing face images with and without masks.
- 1.2 Apply face detection using OpenCV or MediaPipe.
- 1.3 Resize and normalize images for model compatibility.
- 1.4 Perform data augmentation (e.g., rotation, flipping, brightness adjustment) to improve generalization.

FR2 —Model Development

- 2.1 Implement a Convolutional Neural Network (CNN) or fine-tune a pretrained model (such as MobileNet or VGG16).
- 2.2 Train the model to classify images into two categories: *Mask* and *No Mask*.
- 2.3 Use dropout, batch normalization, and hyperparameter tuning to optimize performance.
- 2.4 Target validation accuracy above 95%.

FR3 —User Interface Development

- 3.1 Desktop Application (Tkinter/PyQt):
 - Allow users to upload photos for detection.
 - Display prediction results and confidence scores.
- 3.2 Web Application (Flask/Django):
 - Provide a web-based interface for image uploads or live camera feed detection.

FR4 —Real-Time Detection

- 4.1 Use OpenCV to capture live video from a webcam or CCTV feed.
- 4.2 Detect and classify faces as *Mask* or *No Mask* in real-time.
- 4.3 Display bounding boxes and labels with confidence levels.

FR5 —Database Integration

- 5.1 Store detection results including timestamps, image paths, and predictions using SQLite or MySQL.
- 5.2 Allow administrators to view and analyze detection statistics.

FR6 —Performance Evaluation & Optimization

- 6.1 Evaluate system performance using metrics such as accuracy, precision, recall, and F1-score.
- 6.2 Test under various lighting and camera conditions.
- 6.3 Optimize the model for faster real-time performance while maintaining high accuracy.

Non-Functional Requirements:

NFR1 — Performance

- The system should classify a face in less than **1-3 seconds**.
- The system must detect and classify faces (Mask / No Mask) in **real-time with at least 15 FPS** on a standard webcam.
- The accuracy of predictions should be $\geq 95\%$.

NFR2 — Reliability

- The system should maintain at least **95% uptime** during normal use.
- The system must produce consistent results under varying conditions.
- The model should not crash during real-time video processing.

NFR3 — Accuracy & Quality Requirements

- The trained model should achieve:
 - **Training accuracy $\geq 95\%$**
 - **Validation accuracy $\geq 95\%$**

NFR4 — Usability

- The user interface (Desktop or Web) must be simple, intuitive, and user-friendly for non-technical users.
- Buttons, labels, and messages must be clearly readable and properly placed.
- Users should be able to upload an image or start a webcam session in 1–2 clicks.
- Proper error messages must be displayed for:
 - Invalid image formats
 - Camera access failure
 - No face detected

NFR5 — Scalability

- The system should be able to handle:
 - **Multiple face detections** in a single frame.
 - Growing database entries without performance degradation.

NFR6 — Security

- User-uploaded images must be processed securely.
- Database logs must be protected against unauthorized access.

NFR7 — Maintainability

- The code should follow clean coding practices
- Modules should be separated as:
 - Data preprocessing
 - Real-time detection
 - User interface
 - Database operations
- Comments and documentation should be provided for all major functions.

NFR8 — Portability

- The application must run on:
 - Windows 10 / 11
- The web version should run in all major browsers (Chrome, Firefox, Edge).

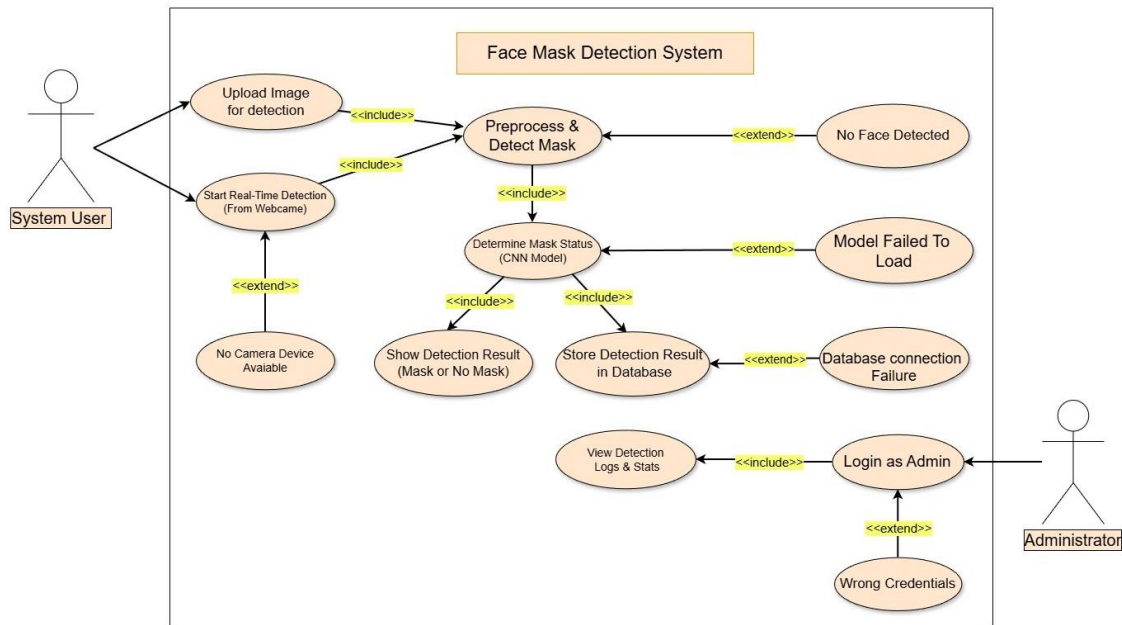
NFR9 — Compatibility Requirements

- The system should be compatible with:
 - Python 3.8+
 - OpenCV, TensorFlow/Keras/PyTorch
 - SQLite/MySQL
 - Must support standard webcam devices without requiring special drivers.
-

NFR10 — Ethical & Privacy Requirements

- The system must not use collected images for any purpose other than mask detection.
- The system must avoid race, age, or gender bias during detection.

Use Case Diagram:



Usage Scenarios:

Item	Description
Use Case Title	Upload Image for Detection
Use Case ID	UC-01
Actor	System User
Description	The user uploads a static image file to the system to check if the person in the image is wearing a face mask or not
Pre-Conditions	System running; image available.
Post-Conditions	Image forwarded for processing.
Main Actions	1. User selects "Upload Image" option 2. System displays file browser dialog 3. User selects image file (JPG/PNG) 4. System validates file format 5. System processes image
Alternative Paths	• User cancels image upload. • User uploads different image after failure. • User chooses wrong file
Exceptions	Invalid format. Corrupted file. File Size exceed than 5 MB
Author	Rehmat Ullah — BC220404931

Item	Description
------	-------------

Item	Description
Use Case Title	Start Real-Time Detection (From Webcam)
Use Case ID	UC-02
Actor	System User
Description	The user activates the webcam for real time detection to detect face masks in real-time.
Pre-Conditions	• A camera device is connected and drivers are installed • User has granted camera permissions.
Post-Conditions	Frames sent for processing to detect mask.
Main Actions	• User clicks real-time detection. • System checks for camera. • Webcam initializes. • Live frames captured.
Alternative Paths	User stops detection manually. Camera disconnects from the system
Exceptions	No camera Device Available → UC-13.
Author	Rehmat Ullah — BC220404931

Item	Description
Use Case Title	Preprocess & Detect Mask
Use Case ID	UC-03
Actor	Internal System
Description	Preprocesses input and performs face detection.
Pre-Conditions	Valid input available.
Post-Conditions	Preprocessed image sent to Determine Mask Status (CNN Model)
Main Actions	Detect face Resize/normalize image
Alternative Paths	None
Exceptions	No face Detected → UC-09.
Author	Rehmat Ullah — BC220404931

Item	Description
Use Case Title	Determine Mask Status (CNN Model)
Use Case ID	UC-04
Actor	Internal System
Description	The core processing unit that analyzes the face data to classify the face status as Mask/No Mask using CNN.
Pre-Conditions	Preprocessed image must exist. CNN model must be loaded.
Post-Conditions	Output available for display & storage. A classification (Mask / No Mask) is returned
Main Actions	• Load CNN model. • Run prediction. • Generate confidence score. • Send output to UI & DB module.
Alternative Paths	No Face
Exceptions	Model Failed to Load→ UC-10.
Author	Rehmat Ullah — BC220404931

Item	Description
Use Case Title	Show Detection Result
Use Case ID	UC-05
Actor	Internal System
Description	System displays the final mask detection status to the user.
Pre-Conditions	Valid classification results available
Post-Conditions	User views final detection output.
Main Actions	• Receive prediction result. • Display mask/no-mask label. • Show confidence score and bounding boxes.
Alternative Paths	None
Exceptions	Missing result.
Author	Rehmat Ullah — BC220404931

Item	Description
Use Case Title	Store Detection Result in Database
Use Case ID	UC-06
Actor	Internal System
Description	Saves mask detection result into database.
Pre-Conditions	Result must be generated.
Post-Conditions	Entry stored successfully.
Main Actions	• Prepare data (status, timestamp etc). • Connect to DB. • Insert detection record.
Alternative Paths	System retries if DB is busy.
Exceptions	Database connection Failure → UC-11.
Author	Rehmat Ullah — BC220404931

Item	Description
Use Case Title	View Detection Logs & Stats
Use Case ID	UC-07
Actor	Administrator
Description	Allows authorized administrators to view historical detection data
Pre-Conditions	Admin logged in.
Post-Conditions	Logs displayed.
Main Actions	• Admin opens logs. • System retrieves entries. • Display logs & stats.
Alternative Paths	None
Exceptions	No logs available in database. DB read error
Author	Rehmat Ullah — BC220404931

Item	Description
Use Case Title	Login as Admin
Use Case ID	UC-08
Actor	Administrator
Description	Admin logs into system for privileged access.
Pre-Conditions	Credentials must exist.
Post-Conditions	Admin panel displayed.
Main Actions	1. Enter username & password. 2. System validates credentials. 3. Admin redirected to dashboard.
Alternative Paths	Retry login
Exceptions	Wrong credentials → UC-12.
Author	Rehmat Ullah — BC220404931

Item	Description
Use Case Title	No Face Detected
Use Case ID	UC-09
Actor	Internal System
Description	Triggered when the system fails to detect any face in the input.
Pre-Conditions	Face detection executed.
Post-Conditions	User informed.
Main Actions	• System runs face detection. • No face detected. • Display error message to user.
Alternative Paths	User retries with new image
Exceptions	None
Author	Rehmat Ullah — BC220404931

Item	Description
Use Case Title	Model Failed to Load
Use Case ID	UC-10
Actor	Internal System
Description	CNN model could not be loaded.
Pre-Conditions	CNN model required. Classification request initiated.
Post-Conditions	Detection halted.
Main Actions	• System attempts to load model. • Loading fails. • System shows failure message.
Alternative Paths	None
Exceptions	None
Author	Rehmat Ullah — BC220404931

Item	Description
Use Case Title	Database Connection Failure
Use Case ID	UC-11
Actor	Internal System
Description	System cannot connect to database.
Pre-Conditions	Store operation triggered.
Post-Conditions	Record not saved.
Main Actions	• System attempts DB connection. • Connection fails. • System shows DB failure message.
Alternative Paths	None
Exceptions	None
Author	Rehmat Ullah — BC220404931

Item	Description
Use Case Title	Wrong Credentials
Use Case ID	UC-12
Actor	Administrator
Description	Triggered when the admin enters invalid login information.
Pre-Conditions	Login attempt made.
Post-Conditions	Access denied.
Main Actions	• Admin submits credentials. • System detects mismatch. • Error message shown.
Alternative Paths	Admin retries login
Exceptions	None
Author	Rehmat Ullah — BC220404931

Item	Description
Use Case Title	No Camera Device Available
Use Case ID	UC-13
Actor	Internal System
Description	Webcam missing when real-time detection starts.
Pre-Conditions	User requested real-time detection.
Post-Conditions	Process stops.
Main Actions	• System searches for camera. • No device detected or camera is busy. • System shows error to user.
Alternative Paths	User connects camera and retries. User upload image using image upload option. System scans for secondary camera.
Exceptions	None
Author	Rehmat Ullah — BC220404931

Adopted Methodology

Waterfall Model

Overview

The waterfall model, also known as the linear sequential model, was the first published model of the software development process. It was derived from other engineering processes. The model is characterized by a cascade from one phase to the next, which is why it is named the "waterfall" model. It proposes a systematic, sequential approach to software development.

Core Concept

The model maps principal stages directly onto fundamental development activities. Development begins at the system level and progresses sequentially through analysis, design, coding, testing, and maintenance. The literature typically identifies between 5 to 8 stages; the five fundamental stages are described below.

The Five Principal Stages

1. Requirement Analysis and Definition (What)

- The system's services, constraints, and goals are established through consultation with users.
- These are then defined in detail to create a formal system specification.

2. System and Software Design (How)

- The system design process allocates requirements to hardware or software systems and establishes an overall system architecture.
- Software design involves defining fundamental system abstractions and their relationships.

3. Implementation and Unit Testing (How)

- The software design is realized as a set of programs or program units.
- Unit testing verifies that each individual unit meets its specifications.

4. Integration and System Testing

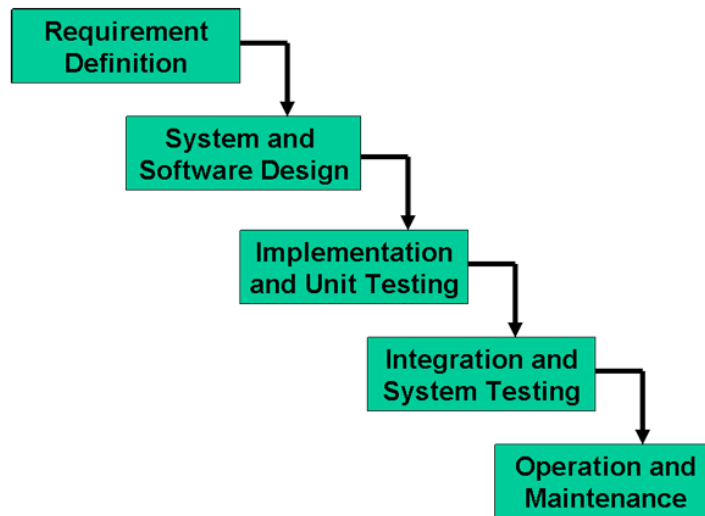
- Individual program units are integrated and tested as a complete system to ensure all software requirements are met.
- Once tested, the software system is delivered to the customer.

5. Operation and Maintenance

- This is typically the longest phase of the software life cycle.
- The system is installed and put into practical use.
- Maintenance involves:
 - Correcting errors not discovered earlier.
 - Improving the implementation of system units.

- Enhancing the system's services as new requirements are discovered.

Waterfall Model Diagram



Spiral Model

The Spiral Model was developed by Barry Boehm. Its core principle is to proactively manage and avert risk, as software development inherently involves numerous risks. Examples include key personnel resigning at a critical point or the software development vendor going bankrupt.

In essence, the model can be seen as an extension of the Waterfall Model, integrating systematic risk analysis into each phase. Each development stage begins with identifying potential alternatives and performing a thorough risk assessment. This is followed by an evaluation of the stage and planning for the next phase. A key feature is that if critical risks cannot be resolved, the project is terminated immediately to prevent further investment in a failing endeavor.

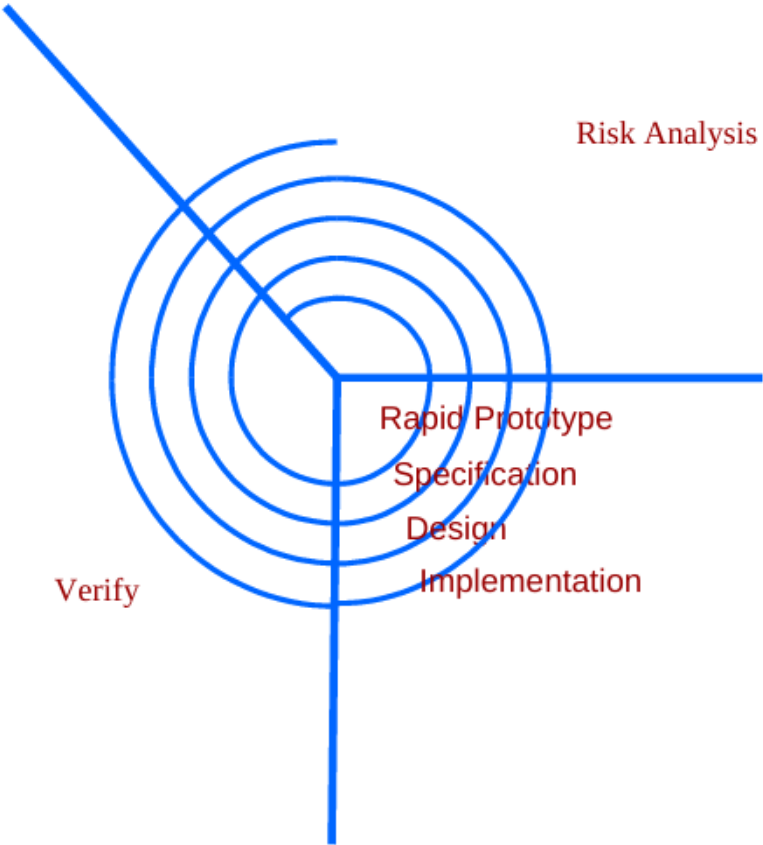
The model is visually represented as a spiral with two key dimensions:

- The **radial dimension** represents the cumulative cost incurred on the project to date.
- The **angular dimension** represents the progress made through the phases of the spiral cycle.

Each cycle (or loop) around the spiral starts by defining the objectives for that phase. Importantly, depending on the needs of that specific phase, a different internal process model (like prototyping or incremental development) can be employed.

The primary strength of the Spiral Model is its acute sensitivity to risk. Its iterative nature allows teams to better judge the extent of testing needed and creates a seamless transition from development to maintenance, effectively blurring the line between the two. However, a noted limitation is that it is generally considered suitable only for large-scale software projects and is most effectively applied to internal, in-house development.

Spiral Model Diagram



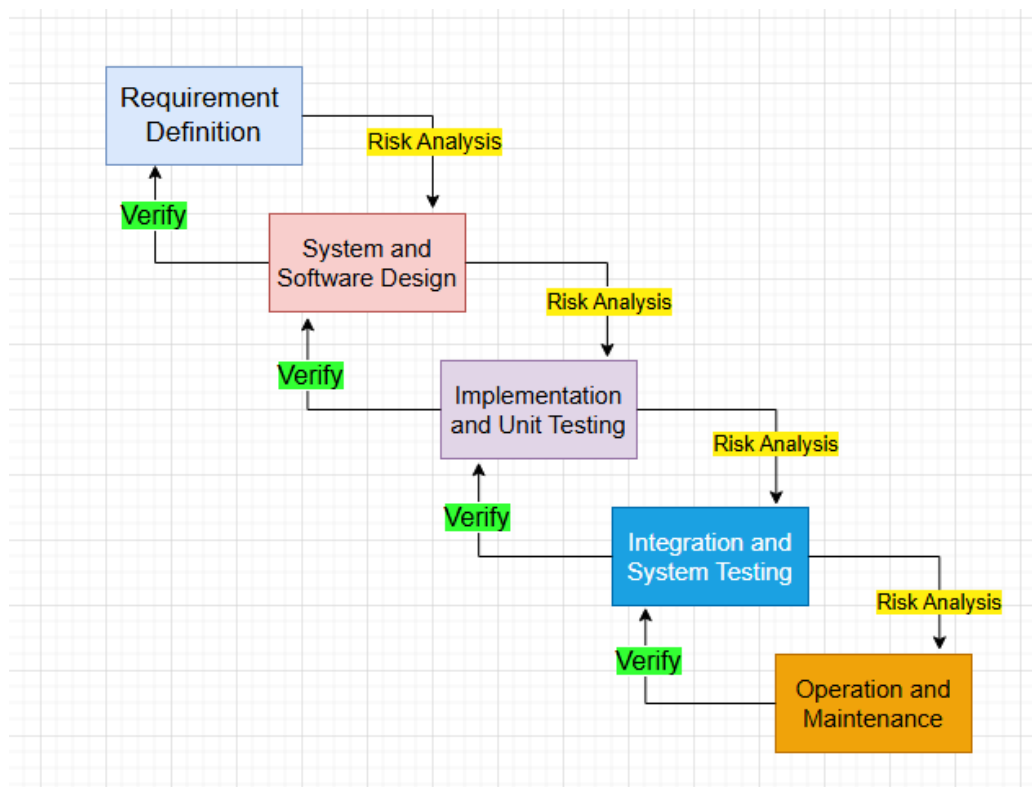
Vu Process Model

For my project, the **Face Mask Detection System**, I have selected the **VU Process Model** as the development methodology. This model is created by combining both the Waterfall and Spiral models, which makes it a good fit for my system because it includes structured steps as well as areas that need continuous improvement, such as machine learning.

The **Waterfall component** of the VU model allows me to follow a clear and organized sequence of phases. First, I gather the requirements of the system, then I design the architecture, develop the mask detection model, implement real-time detection features, perform testing, and finally deploy the complete system. This step-by-step approach keeps the development process well-planned and systematic.

The **Spiral component** helps me review and enhance the system whenever necessary. Since the deep learning model may require retraining for better accuracy, this part of the methodology gives me the flexibility to revisit previous stages, improve the dataset, adjust the model, and reduce potential risks. Because of this iterative nature, the system becomes more reliable and accurate over time.

Overall, the VU Process Model provides both structure and flexibility, which suits the technical requirements of my Face Mask Detection System.



Work Plan (Use MS Project to create Schedule/Work Plan)

